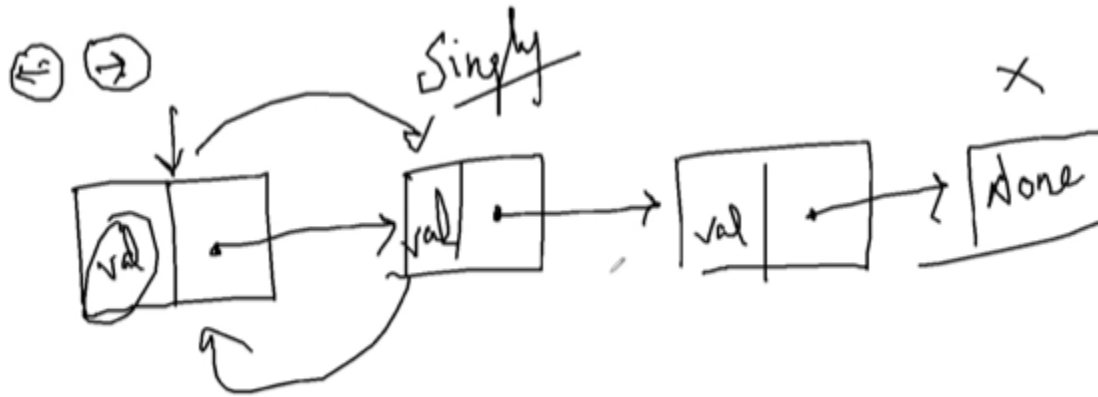


LINKED LIST

Eg : .google ($\leftarrow \rightarrow$) => eg of singly linked list



Node creation

Blueprint or skeletal

all functions inside will have self as param

class node:

```
def __init__(self,x):
```

```
    self.val=x
```

```
    self.next=None
```

class LinkedList:

```
def __init__(self):
```

```
    self.head = None
```

```
    self.last = None
```

```
    self.lenngh = 0
```

def addNode(self,x):

```
    newNode = node(x)
```

```
    if self.head == None:
```

```
        self.head = newNode # first and last same
```

```

        self.last = newNode
    else:
        self.last.next = newNode
        self.last = self.last.next
    self.lennngth+=1

    print("Succesfully added new Node!")

    # newNode
def addMiddle(self,where,val): # temp temp2
    self.lennngth+=1
    temp=None
    cur=self.head
    while cur!=None:
        if cur.val == where:
            temp = cur
            cur=cur.next
    if temp==None or temp.next==None:
        newNode = node(val)
        self.last.next = newNode
        self.last = self.last.next
    else:
        newNode = node(val)
        temp2 = temp.next
        temp.next = newNode
        newNode.next = temp2

```

```
print("Successfully Inserted!!")
```

```
def printLL(self):
```

```
    cur = self.head
```

```
    while cur!=None:
```

```
        print(cur.val) # 5 10 100
```

```
        cur = cur.next
```

```
    print("Printed Successfully")
```

```
def length(self):
```

```
    print("The length of LL is",self.lennngth)
```

```
    return self.lennngth
```

```
def deleteFirst(self):
```

```
    if self.head==None:
```

```
        print("The Linked List is empty!")
```

```
        return
```

```
    self.head = self.head.next
```

```
    print("The first element is deleted!")
```

```
def delNodeAtLast(self):
```

```
    last = self.head
```

```
    if last == None:
```

```
        print("The LinkedList is empty!")
```

```
        return
```

```
    if last.next == None:
```

```
        self.head = None
```

```
        print("Last Node Successfully Deleted!")
```

```
        self.lennngth-=1
```

```

        return

    while last.next.next!=None:

        last = last.next

    last.next = None

    self.lennngth-=1

    print("Last Node Successfully Deleted!")

def insertFirst(self,x):

    newNode = node(x)

    if self.head == None:

        self.head = newNode

        self.last = newNode

    else:

        newNode.next = self.head

        self.head = newNode

    self.lennngth+=1

    print("Successfully Inserted!")

def deleteMiddle(self,val):

    temp = self.head

    prev = None

    while temp.next != None:

        if temp.val == val:

            prev.next = temp.next

            self.lennngth-=1

            print(prev.val," ",temp.next.val)

            print("Successfully Deleted!")

            return

```

```

        else:

            prev = temp

            temp = temp.next

        print("The element doesnt exist!")

    def reverse(self):

#           c
#   # None<-1 <-2<-3<-4-> 5-> 6   # cur,prev,nex
#           n
#           p

        prev = None, nex=None

        cur = head

        while cur!=None:

            nex = cur.next

            cur.next = prev

            prev = cur

            cur = nex

# calling all func

LL = LinkedList()

while True:

    n = int(input("Enter 1.Insertion 2.Printing 3.Delete_Last 4.Length 5.InsertFirst 6.AddMiddle

                    7.Delete_First 8.Delete_middle"))

    if n==1:

        print("Enter value for Node")

        val=int(input())

        LL.addNode(val)

    elif n==2:

```

```
LL.printLL()
elif n==3:
    LL.delNodeAtLast()
elif n==4:
    LL.length()
elif n==5:
    print("Enter value for Node to insert first")
    val=int(input())
    LL.insertFirst(val)
elif n==6:
    print("Enter value for Node to insert middle")
    val=int(input())
    print("Enter value after which/where")
    where=int(input())
    LL.addMiddle(where,val)
elif n==7:
    LL.deleteFirst()
elif n==8:
    print("Enter value for Node to delete middle")
    val=int(input())
    LL.deleteMiddle(val)
else:
    break
```

1. Creating a class node

```
# Node create, LL done class!

# Node creation
# Blueprint or skeletal
# all functions inside will have self as param
class node:
    def __init__(self,x):
        self.val=x
        self.next=None
        #print("Node Successfully created with value",self.val)
    def add(self):
        print("Hey Athul")
    def sub(self):
        print("Hey Anjana")
```

2. Creating objects of class node

```
node1 = node(5) # Node created
node2 = node(10) # Node created
node3 = node(100) # Node created
```

__init__(self) is called every time a node is created
add(self), sub(self) -> has to be called for it to be executed

3. To link the nodes created

```
node1.next = node2
node2.next = node3
```

node1 -> node2 -> node3

By default node3 is assigned with None (self.next = None)

4. To traverse the LL

```

node1.next = node2
node2.next = node3
head = node1

while head!=None:
    print(head.val) # 5 10 100
    head = head.next
print(" The LL is Done!")
# node1 -> node2 -> node3

```

5. Naive length algorithm (takes $O(N)$ time to find the length of array)

```

def naive_length(self):
    cur = self.head
    count=0
    while cur!=None:
        #print(cur.val) # 5 10 100
        cur = cur.next
        count+=1

```

6. To get len of LL in $O(1)$

Class Node: # declare the length =0 in as a class node parameter (global var)

```

Self.head = None
Self.last = None
Self.length = 0

```

Def addNode(self , x): # add the len by 1 when adding a node

```

Self.length += 1

```

Def deleteNode(self): # delete len by 1 when deleting a node

```

Self.length -= 1

```

#optimal way to find the len

```

def length(self):
    print("The length of LL is",self.lennngth)
    return self.lennngth

```

Prev : insert mid


```

def addMiddle(self, where, val):    # temp temp2
    self.length+=1
    temp=None
    cur=self.head
    while cur!=None:
        if cur.val == where:
            temp = cur
            cur=cur.next

    if temp==None or temp.next==None:
        newNode = node(val)
        self.last.next = newNode
        self.last = self.last.next
    else:
        newNode = node(val)
        temp2 = temp.next
        temp.next = newNode
        newNode.next = temp2

    print("Successfully Inserted!!")

```

23-01-2024

Topics : deleteFirst , deleteMiddle , reverse

1.deleteFirst

```

def deleteFirst(self):
    if self.head==None:
        print("The Linked List is empty!")
        return
    self.head = self.head.next
    print("The first element is deleted!")

```

```

elif n==7:
    LL.deleteFirst() # call the func

```

2.deleteMiddle

```

# Pointer
# while cur:
# if self.head!=None | if self.head
def delMiddle(self,val):
    # Corner
    if self.head and self.head.val == val: #
        LL.deleteFirst()
    elif self.last and self.last.val == val:
        LL.delNodeAtLast()
    else:
        temp = self.head
        prev = None
        while temp.next != None:
            if temp.val == val:
                prev.next == temp.next
                return
            else:
                prev = temp
                temp = temp.next

```

p t
1 2 3 4 5 6 7

Note:

Self.head != None (same as) self.head.

using next.next

```

self.length-=1
print("Successfully Deleted!")
else:
    temp = self.head
    while temp and temp.next!=None:
        if temp.next.val == val:
            temp.next=temp.next.next

```

t
1 2 3 4 5 6 7

3.def Reverse() → singly LL

Method 1:

[Naive approach # takes extra linked list → [O(n) space]

Use a array -[1,2,3,4,5] (get the val of LL)

Insert the ele of array in the reverse order → to new LL or overwrite the existing LL

Method 2:

```
def reverse(self):

    # None <- 1 <- 2 <- 3 <- 4 <- 5 <- 6      # cur, prev, nex
    #      | p      n
    #      | p      n

    prev = None, nex=None
    cur = head
    while cur!=None:
        nex = cur.next
        cur.next = prev
        prev = cur
        cur = nex
```

leet code Problems

1.Middle of LL

<https://leetcode.com/problems/middle-of-the-linked-list/submissions/1155677343/>

Floyd's algorithm

1. **Fast** and **slow pointer** approach → harry and tortoise method
2. **Slow** : moves **one step**
3. **Fast** : moves **two steps**

Algorithm

The steps of Floyd's algorithm are shown below:

- Take two pointers and label them as fast and slow.
- Point out both pointers to the head of the linked list.
- Move the slow pointer by one node and the fast pointer by two nodes while traversing a linked list.
- The loop is detected if both pointers meet at the same node.
- If both pointers do not meet at the same node and the fast pointer reaches the end of the linked list, then there is no loop in the linked list.

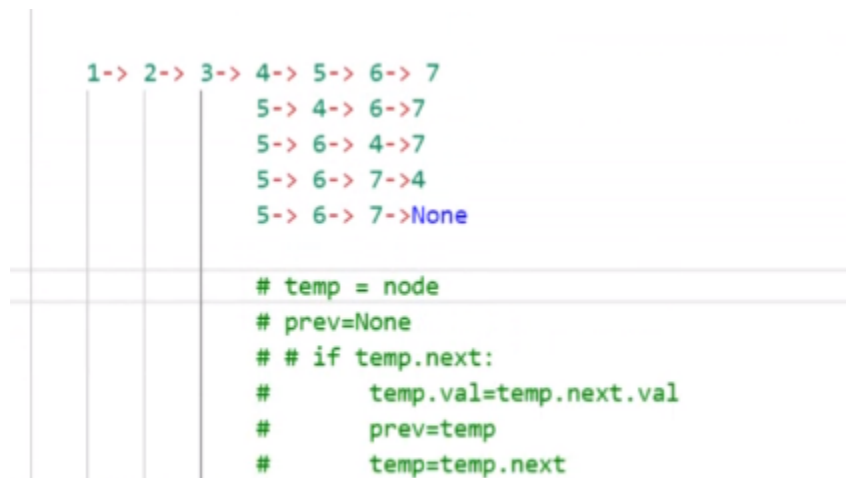
```
slow = head
fast = head
while fast and fast.next:
    slow = slow.next
    fast = fast.next.next
return slow
```

2.Head Node is not given , to Delete a node

<https://leetcode.com/problems/delete-node-in-a-linked-list>

```
def deleteNode(self, node):  
    curr = node  
    prev = None  
    while curr.next:  
        curr.val = curr.next.val  
        prev = curr  
        curr = curr.next  
    prev.next = None
```

Illustrations:



3.Linked List Cycle [floyd's algorithm]

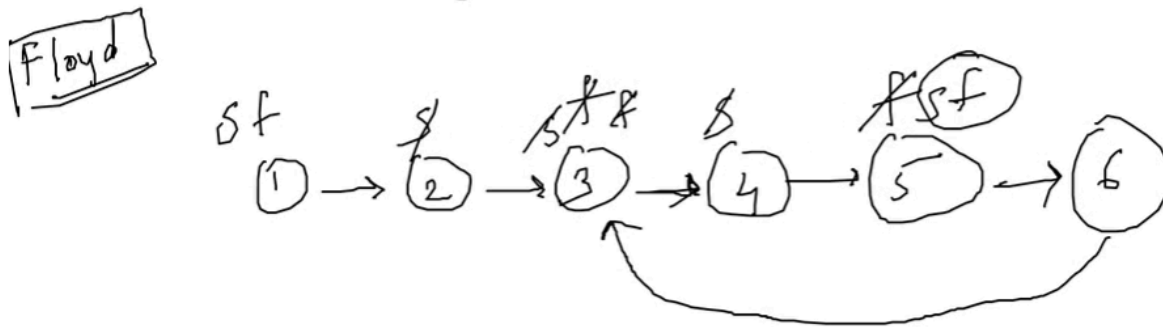
<https://leetcode.com/problems/linked-list-cycle/>

```
def hasCycle(self, head: Optional[ListNode]) -> bool:  
    slow = head  
    fast = head  
    while fast and fast.next:  
        slow = slow.next  
        fast = fast.next.next  
    if fast == slow:  
        return True  
    return False
```

Core logic :

1. fast == slow → there is a cycle

Illustration :



1. Merge Two Sorted Lists

<https://leetcode.com/problems/merge-two-sorted-lists/>

```
l=[]
while list1:
    l.append(list1.val)
    list1 = list1.next

while list2:
    l.append(list2.val)
    list2 = list2.next

l = sorted(l)
head = ListNode(-1) #dummy node
temp = head
while l:
    val = l.pop(0)
    newNode = ListNode(val) # to create new node
    head.next = newNode
    head = head.next

return temp.next # leave the dummy head (-1) and return the
                # head to print LL
```

1. Set temp = head # to print the LL
2. Create a dummy head -1
3. And return the LL by return head without dummy

Illustration

```
# h
# -1->1->1->2->3->4
# d
# nlogn ---> O(n)
# O(n) ---> O(n)
```

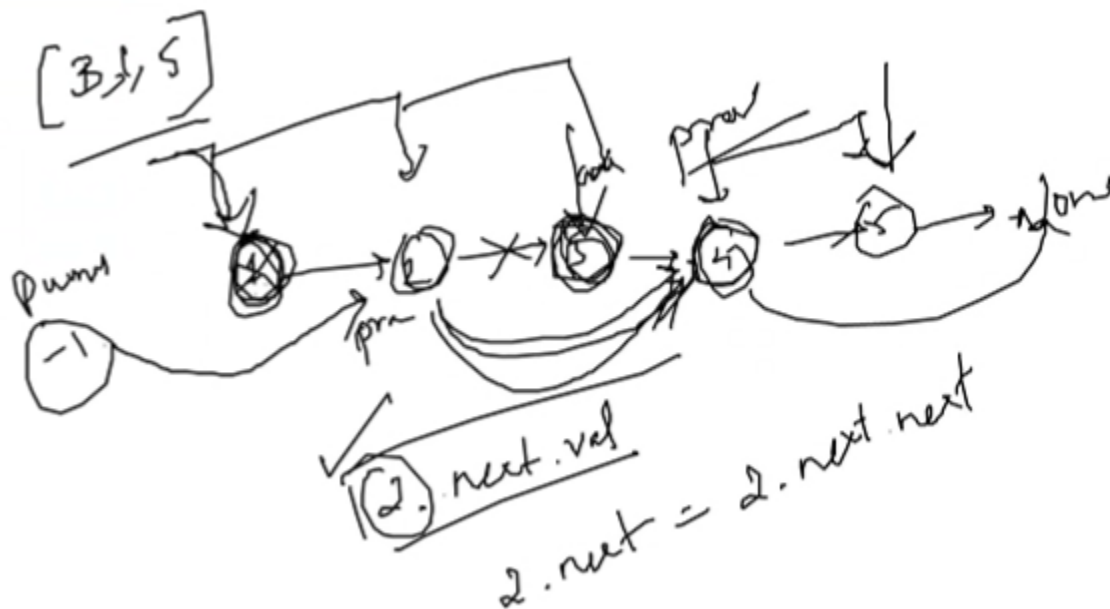
To optimize to

1. Time complexity $\rightarrow O(n)$
2. Space complexity $\rightarrow O(1)$

[5, 1, 2, 3, 4]

1 2 3 4 5 n=5 n-k = 3

3217. Delete Nodes From Linked List Present in Array



```
""" Solution:
def modifiedList(self, nums: List[int], head: Opt

    s = set(nums)

    cur = head
    dum = head1 = ListNode(-1)
    dum.next = head

    while dum and dum.next:
        if dum.next.val in s:
            dum.next = dum.next.next
        else:
            dum = dum.next

    return head1.next
```